

AgroLang: Automação Declarativa e Tradução de Regras para Sistemas de Monitoramento Agrícola Inteligente

Victor Augusto de Oliveira
Engenharia da Computação
FHO – Fundação Herminio Ometto
Araras, Brasil
victoroliveira855@alunos.fho.edu.br

Abstract—Este artigo descreve o desenvolvimento da AgroLang, uma Linguagem de Domínio Específico (DSL) projetada para otimizar o manejo agrícola em pequenas propriedades. O compilador traduz regras declarativas em objetos estruturados (JSON), permitindo a automação de alertas e sistemas de irrigação baseados em dados de sensores IoT e visão computacional, sem a necessidade de reprogramação de hardware.

Index Terms—DSL, Compiladores, Agricultura de Precisão, IoT, AgroLang, Python

I. INTRODUÇÃO

A agricultura familiar enfrenta o desafio constante de gerenciar recursos hídricos e infestações de pragas com precisão. Soluções recentes, como o Sistema Inteligente de Monitoramento Agrícola desenvolvido por Oliveira [6], demonstram a viabilidade de integrar microcontroladores ESP32 e redes neurais (YOLOv8) para coletar dados vitais do campo a baixo custo. No entanto, embora essas arquiteturas forneçam dados valiosos, a lógica de decisão para disparar alertas ou ativar atuadores no firmware é frequentemente estática e de difícil alteração pelo usuário final [6].

A AgroLang surge como solução de software para abstrair essa complexidade, operando como a camada de decisão inteligente do sistema proposto em [6]. Ela oferece uma interface declarativa onde o produtor pode definir comportamentos complexos, como irrigação por demanda ou alertas de fitossanidade, de forma dinâmica e segura, sem reprogramar os módulos físicos ou o backend da plataforma web.

II. FUNDAMENTAÇÃO TEÓRICA

O compilador AgroLang fundamenta-se na teoria de **Gramáticas Livres de Contexto (GLC)**, utilizando as etapas clássicas de tradução para garantir a integridade das instruções. A análise léxica e sintática é realizada por meio da biblioteca **PLY (Python Lex-Yacc)**, que permite a construção de uma **Árvore Sintática Abstrata (AST)** para representar a hierarquia das regras [1].

Um diferencial desta versão é a inclusão da análise semântica para validação de coerência lógica e a geração de código em formato **JSON**, servindo como representação intermediária independente de plataforma [5].

A AgroLang diferencia-se ao propor uma gramática leve que se integra nativamente tanto com a telemetria do solo (Módulo 1 de [6]) quanto com algoritmos de Visão Computacional (Módulo 2 de [6]) executados na extremidade da rede (*Edge Computing*).

III. DESENVOLVIMENTO / METODOLOGIA

A. Detalhamento da Gramática

A AgroLang foi projetada segundo os princípios de GLC, com foco em legibilidade declarativa para o domínio agrícola. A seguir apresenta-se o detalhamento completo de seus componentes léxicos, sintáticos e semânticos.

1) **Categorias de Tokens:** O analisador léxico reconhece seis categorias de tokens, implementadas via expressões regulares no PLY:

- 1) **Palavras Reservadas:** delimitam a estrutura de uma regra e os operadores lógicos: REGRA, SE, ENTÃO, SENÃO, E, OU.
- 2) **Identificadores de Sensores:** variáveis numéricas mapeadas diretamente aos módulos físicos, conforme Tabela I.
- 3) **Funções de Ação:** tokens especiais reconhecidos antes do padrão genérico de identificador para evitar ambiguidade léxica: `disparar_alerta` e `irrigar`.
- 4) **Literais Numéricos:** inteiros ou decimais reconhecidos por `\d+(\.\d+)?` e convertidos automaticamente para `int` ou `float`.
- 5) **Literais de String:** sequências entre aspas duplas com suporte a caracteres escapados; as aspas delimitadoras são descartadas pelo lexer.
- 6) **Operadores e Delimitadores:** relacionais (`>`, `<`, `==`, `>=`, `<=`, `!=`) e estruturais (`{`, `}`, `(`, `)`, `;`).

2) **Gramática BNF Completa:** A gramática formal da AgroLang, derivada diretamente das regras de produção implementadas no parser LALR(1), é definida como:

TABLE I
VARIÁVEIS DE SENSORES E SEUS MÓDULOS DE ORIGEM

Variável	Módulo de Origem
umidade_solo	Módulo 1 – Solo
temperatura_solo	Módulo 1 – Solo
ph_solo	Módulo 1 – Solo
nivel_agua	Módulo 1 – Solo
umidade_ar	Módulo 1 – Ar
temperatura_ar	Módulo 1 – Ar
luminosidade	Módulo 1 – Ambiente
contagem_pragas	Módulo 2 – Visão Computacional
indice_saude	Módulo 2 – Visão Computacional

TABLE II
TIPOS DE NÓS DA AST E SEUS ATRIBUTOS

Tipo	Atributos	Origem
programa	regras: list	Raiz
regra	nome, condicao, acao, acao_senao	REGRA
condicao_logica	operador, esquerda, direita	E / OU
condicao_simples	variavel, operador, valor	expr. relacional
acao_disparar	mensagem	disparar_alerta
acao_irrigar	zona, duracao_minutos	irrigar

$\langle prog \rangle ::= \langle lista \rangle$
 $\langle lista \rangle ::= \langle lista \rangle \langle regra \rangle \mid \langle regra \rangle$
 $\langle regra \rangle ::= REGRA \langle str \rangle \{ SE \langle cond \rangle ENTÃO \langle acao \rangle \}$
 $\quad \mid REGRA \langle str \rangle \{ SE \langle cond \rangle ENTÃO \langle acao \rangle SENÃO \langle acao \rangle \}$
 $\langle cond \rangle ::= \langle cond \rangle E \langle cond \rangle \mid \langle cond \rangle OU \langle cond \rangle$
 $\quad \mid \langle id \rangle \langle op_rel \rangle \langle num \rangle$
 $\langle op_rel \rangle ::= > \mid < \mid == \mid >= \mid <= \mid !=$
 $\langle acao \rangle ::= disparar_alerta(\langle str \rangle);$
 $\quad \mid irrigar(\langle str \rangle; \langle num \rangle);$

A produção de $\langle cond \rangle$ é recursiva, permitindo encadeamento arbitrário de predicados com E e OU. O parser LALR(1) do PLY resolve ambiguidades de associatividade pela ordem das produções.

3) *Estrutura Sintática das Ações*: A linguagem define duas ações de saída com assinaturas distintas:

- **disparar_alerta(msg)**: recebe um argumento STRING e publica uma notificação no dashboard via MQTT, pelo feed agrolang-alertas do Adafruit IO.
- **irrigar(zona; duracao)**: recebe uma STRING para a zona física e um NUMERO para a duração em minutos. O separador ; foi escolhido intencionalmente para evitar conflito léxico com o terminador de instrução. No hardware, essa ação aciona o relé do ESP32 pelo tempo especificado.

O Código 1 apresenta uma regra completa:

```

1 REGRA "Manejo_Setor_A" {
2     SE umidade_solo < 30 E temperatura_ar > 28
3     ENTÃO irrigar("Setor_A"; 15);
4     SENÃO disparar_alerta("Solo_em_niveis_ideais"
5     );
6 }

```

Listing 1. Regra com condicao composta e bloco SENÃO

4) *Representação Interna: AST*: Cada nó da AST é representado pela classe NoAST, que armazena o tipo e os atributos semânticos como campos de objeto. A Tabela II descreve os seis tipos de nós gerados pelo parser.

A hierarquia gerada para o Código 1 é:

```

1 programa
2   regra: "Manejo Setor A"
3     condicao_logica (E)
4       condicao_simples: umidade_solo < 30
5       condicao_simples: temperatura_ar > 28
6   acao: acao_irrigar
7   acao_senao: acao_disparar_alerta
8   mensagem: "Solo em niveis ideais"
9

```

Listing 2. AST do Código 1 (representação textual)

5) *Restrições Semânticas*: Após a construção da AST, o AnalisadorSemantico realiza uma travessia em profundidade validando três invariantes:

- 1) **Unicidade de nomes**: nenhuma regra pode ter o mesmo identificador de outra no mesmo arquivo.
- 2) **Integridade de variáveis**: todo identificador em $\langle condicao_simples \rangle$ deve pertencer ao conjunto fixo da Tabela I. Variáveis não reconhecidas são rejeitadas com listagem das opções válidas.
- 3) **Validade de parâmetros**: a duração em irrigar() deve ser estritamente positiva (> 0), evitando comandos inválidos ao atuador físico.

Somente após a aprovação nas três verificações a execução avança para a geração de código.

6) *Geração de Código: AST $\rightarrow$ JSON*: O código de saída é um objeto JSON consumido pelo servidor Apache, pelo dashboard web e pelo firmware do ESP32 (via ArduinoJson). Cada nó da AST é serializado recursivamente pelas funções _condicao_para_dict e _acao_para_dict. O JSON resultante para o Código 1 é:

```

1 {
2   "regras": [{
3     "nome": "Manejo Setor A",
4     "condicao": {
5       "tipo": "logica", "operador": "E",
6       "esquerda": {
7         "tipo": "comparacao",
8         "variavel": "umidade_solo",
9         "modulo": "Modulo 1 - Solo",
10        "operador": "<", "valor": 30
11      },
12      "direita": {
13        "tipo": "comparacao",
14        "variavel": "temperatura_ar",
15        "modulo": "Modulo 1 - Ar",
16        "operador": ">", "valor": 28
17      }
18    }
19  }]
20 }

```

```

17     }
18   },
19   "entao": {
20     "tipo": "irrigar",
21     "zona": "Setor A",
22     "duracao_minutos": 15
23   },
24   "senao": {
25     "tipo": "disparar_alerta",
26     "mensagem": "Solo em niveis ideais"
27   }
28 }
29 }

```

Listing 3. JSON gerado pelo compilador AgroLang

O campo "modulo" em cada nó *comparacao* é um metadado que permite ao dashboard exibir a procedência física de cada variável sem consultas adicionais ao banco de dados.

B. Ferramentas Utilizadas

O sistema foi implementado em **Python**, aproveitando a compatibilidade com os scripts de IA do Módulo 2 do TCC (YOLOv8). A biblioteca PLY gerou o *lexer* (via expressões regulares) e o *parser* (LALR), garantindo eficiência no processamento de múltiplas regras simultâneas.

C. Arquitetura do Compilador

A arquitetura é dividida em cinco módulos sequenciais:

- 1) **Lexer:** converte o código-fonte em tokens específicos do domínio agrícola.
- 2) **Parser:** valida a gramática e constrói a AST.
- 3) **Analizador Semântico:** verifica nomes duplicados, variáveis desconhecidas e parâmetros de irrigação inválidos (≤ 0).
- 4) **Gerador de Código:** traduz a AST em JSON com mapeamento dos módulos de origem.
- 5) **Interface Principal:** função `compilar()` que orquestra os quatro módulos e retorna o JSON final.

IV. RESULTADOS E DISCUSSÃO

A validação da AgroLang culminou no desenvolvimento de um Ambiente de Desenvolvimento Integrado (IDE) próprio, construído utilizando a biblioteca gráfica *tkinter* da linguagem Python. Esta interface gráfica foi desenhada para abstrair completamente o processo de compilação em linha de comando, oferecendo um editor de texto interativo com realce de sintaxe (*syntax highlighting*) em tempo real. O realce visual diferencia palavras reservadas, variáveis de sensores, valores numéricos e funções de atuação, reduzindo a curva de aprendizado para profissionais da agronomia e produtores rurais.

Para testar a eficiência do compilador e a robustez da análise semântica, o seguinte cenário de manejo integrado foi submetido à IDE:

```

1 REGRA "Alerta_de_Estresse_Hidrico_e_Pragas" {
2   SE umidade_solo < 30.0 E temperatura_ar >
3     35.0
4   ENTAO irrigar("Setor_B"; 20);
5   SENAO disparar_alerta("Condicoes_de_solo_adequadas.");
6 }

```

Listing 4. Regra de teste submetida à IDE

Durante a execução, o *Lexer* e o *Parser* processaram o texto perfeitamente, enquanto o Analizador Semântico validou a existência das variáveis `umidade_solo` e `temperatura_ar` dentro do escopo de sensores permitidos pelo sistema. O sistema demonstrou robustez ao bloquear compilações de testes secundários que tentavam referenciar variáveis não cadastradas ou aplicar tempos negativos de irrigação.

O resultado final do processo de compilação da regra acima foi a geração imediata de um arquivo de configuração no formato JSON. Este objeto estruturado atua como a linguagem intermediária universal do projeto. Na prática, este JSON está pronto para ser despachado via protocolo MQTT para a plataforma web central ou diretamente para os atuadores ligados ao microcontrolador ESP32 no campo, ativando a bomba de água por 20 minutos ou enviando a notificação de conformidade ao operador responsável.

V. CONCLUSÃO

O desenvolvimento da AgroLang e de sua IDE associada cumpriu o objetivo de preencher a lacuna de usabilidade nos sistemas de monitoramento agrícola inteligente. Ao aliar a teoria de construção de compiladores (utilizando a biblioteca PLY) a uma interface gráfica amigável em Python, o projeto permitiu que a lógica de negócio agrícola fosse totalmente desacoplada da complexidade de baixo nível do hardware.

A DSL garante que a orquestração de dados complexos — sejam eles telemétricos (sensores de solo via ESP32) ou baseados em Inteligência Artificial (contagem de pragas via visão computacional) — possa ser gerenciada através de regras declarativas simples, lidas e escritas em português.

Como trabalhos futuros, propõe-se a integração bidirecional direta da IDE AgroLang com o servidor broker MQTT do sistema de monitoramento principal, permitindo que a compilação e a implantação (*deploy*) das regras nos microcontroladores ocorram com apenas um clique. Além disso, a expansão do vocabulário da linguagem para englobar comandos preditivos baseados no histórico de dados pode elevar ainda mais o grau de autonomia da agricultura de precisão para pequenos produtores.

O código-fonte completo da AgroLang, incluindo o compilador e a IDE, está disponível publicamente em: <https://github.com/feedieback/AgroLang>.

REFERENCES

- [1] M. Mernik, J. Heering, and A. M. Sloane, "When and how to develop domain-specific languages," *ACM Computing Surveys*, vol. 37, no. 4, pp. 316–344, 2005.
- [2] W. Groeneveld *et al.*, "A domain-specific language framework for farm management information systems in precision agriculture," *Precision Agriculture*, vol. 22, pp. 1–25, 2021.
- [3] J. Pérez *et al.*, "A Domain Specific Language Proposal for Internet of Things Oriented to Smart Agro," *IEEE Access*, 2023.
- [4] A. García *et al.*, "SimulateIoT: Domain Specific Language to Design, Code Generation and Execute IoT Simulation Environments," *IEEE Access*, vol. 9, pp. 1–15, 2021.
- [5] D. Beazley, *PLY (Python Lex-Yacc)*. Disponível em: <https://github.com/dabeaz/ply>.
- [6] V. A. Oliveira, "Sistema Inteligente de Monitoramento Agrícola," Trabalho de Conclusão de Curso, FHO – Fundação Herminio Ometto, Araras, Brasil. Disponível em: <https://github.com/feedieback/TCC>.